Assignment 1

Green Prompt Engineering

Reducing tokens, time and energy for code docstring generation with small language models

Submitted by	Nishad Ahamed
Course	ENSF 619
Date	October 2026
Code	github.com/ahmd-nish/ensf619-assignments

Table of Contents

1. Experimental Setup

How each metric was measured

2. Results Summary

3. Green Engineering Justification

3.1 Optimal Prompt Design for Green Inference

3.2 Impact of Quantization & Hardware

3.3 Model Robustness and Determinism

3.4 Overall Green Recommendation

Appendix A: Prompt Strings

Submitted files

1. Experimental Setup

The goal was to find the cheapest way, in tokens, time and energy, to get a correct Google-style docstring for the `below_zero` function. I compared three prompts across three small language models, and tested one model at three quantization levels.

Item	Details
Task	Add a Google-style docstring to `below_zero` and return the complete function
Models	Qwen2.5-Coder-1.5B-Instruct, Deepseek-Coder-1.3B-Instruct, SmolLM2-1.7B-Instruct
Quantization (Qwen 1.5B)	FP16 (unquantized), INT8 (bitsandbytes LLM.int8), INT4 (bitsandbytes NF4)
Cloud hardware	NVIDIA RTX A6000 48 GB on a rented Vast.ai instance, used instead of Colab
Local hardware	Apple M2 Pro laptop, 16 GB, PyTorch MPS backend (Qwen FP16, all three prompts)
Runs	18 combinations × 5 epochs = **90 runs** (75 on the cloud GPU, 15 on the laptop)
Decoding	Greedy (no sampling), up to 512 new tokens, one warm-up run per model that was not counted
Software	PyTorch 2.14 / CUDA 13, Transformers 5.21, bitsandbytes 0.50

How each metric was measured

- **T_in:** length of the tokenized prompt, including the model's chat template.
- **T_out:** total output length minus T_in.
- **T_total:** T_in + T_out.
- **Inference time:** `time.perf_counter()` around `model.generate()`, with a device sync before and after.
- **GPU energy:** the NVIDIA energy counter (NVML), read before and after each run. Cloud GPU only.
- **Accuracy (P/F):** every output was checked automatically and then reviewed by hand. A run passes only if:
  - the code is valid Python,
  - the docstring sits *inside* the function,
  - the function body is unchanged and still gives the right answers on 6 test inputs,
  - the docstring describes `operations` and the `bool` return value, and
  - any examples in the docstring are correct.
- **Efficiency ratio (ER):** accuracy rate ÷ average T_total × 1,000, which means the number of correct docstrings per 1,000 tokens.

2. Results Summary

All values are averages over 5 epochs, except time and energy, which are medians. Medians are used because a few cloud runs were slowed down by other activity on the shared machine.

Model	Quant	Prompt	T_in	T_out	T_total	Time (s)	Time SD (s)	Energy (J)	Correct	ER
Qwen 1.5B	FP16	P-1	185	124	309	3.73	1.501	395	0/5	0.00
Qwen 1.5B	FP16	**P-2**	151	129	**280**	**3.82**	0.040	**399**	**5/5**	**3.57**
Qwen 1.5B	FP16	P-3	142	108	250	3.16	0.018	338	0/5	0.00
Qwen 1.5B	INT8	P-1	185	122	307	19.52	4.908	1,567	0/5	0.00
Qwen 1.5B	INT8	P-2	151	147	298	33.35	15.950	2,623	5/5	3.36
Qwen 1.5B	INT8	P-3	142	122	264	19.46	20.782	1,579	5/5	3.79
Qwen 1.5B	INT4	P-1	185	119	304	5.90	0.201	527	0/5	0.00
Qwen 1.5B	INT4	P-2	151	149	300	7.32	7.349	666	5/5	3.33
Qwen 1.5B	INT4	P-3	142	108	250	5.70	0.444	504	0/5	0.00
Deepseek 1.3B	FP16	P-1	237	238	475	5.89	0.177	627	5/5	2.11
Deepseek 1.3B	FP16	P-2	203	158	361	3.95	0.047	416	0/5	0.00
Deepseek 1.3B	FP16	P-3	190	257	447	6.28	0.101	677	5/5	2.24
SmolLM2 1.7B	FP16	P-1	186	150	336	3.76	0.076	443	5/5	2.98
SmolLM2 1.7B	FP16	P-2	152	143	295	3.54	0.037	411	5/5	3.39
SmolLM2 1.7B	FP16	P-3	140	145	285	3.55	0.063	427	5/5	3.51
Qwen 1.5B (M2 Pro)	FP16	P-1	185	124	309	3.86	1.012	n/a	0/5	0.00
Qwen 1.5B (M2 Pro)	FP16	P-2	151	129	280	3.14	1.558	n/a	5/5	3.57
Qwen 1.5B (M2 Pro)	FP16	P-3	142	108	250	3.46	2.086	n/a	0/5	0.00

Energy is GPU board energy per run, from NVML. It could not be measured on the laptop. The full per-epoch data (90 rows) is in `results/experimental_matrix.xlsx`.

3. Green Engineering Justification

3.1 Optimal Prompt Design for Green Inference

> **Lowest average total tokens: P-3** (299 tokens). **Best efficiency ratio: P-2** (2.61 correct docstrings per 1,000 tokens).

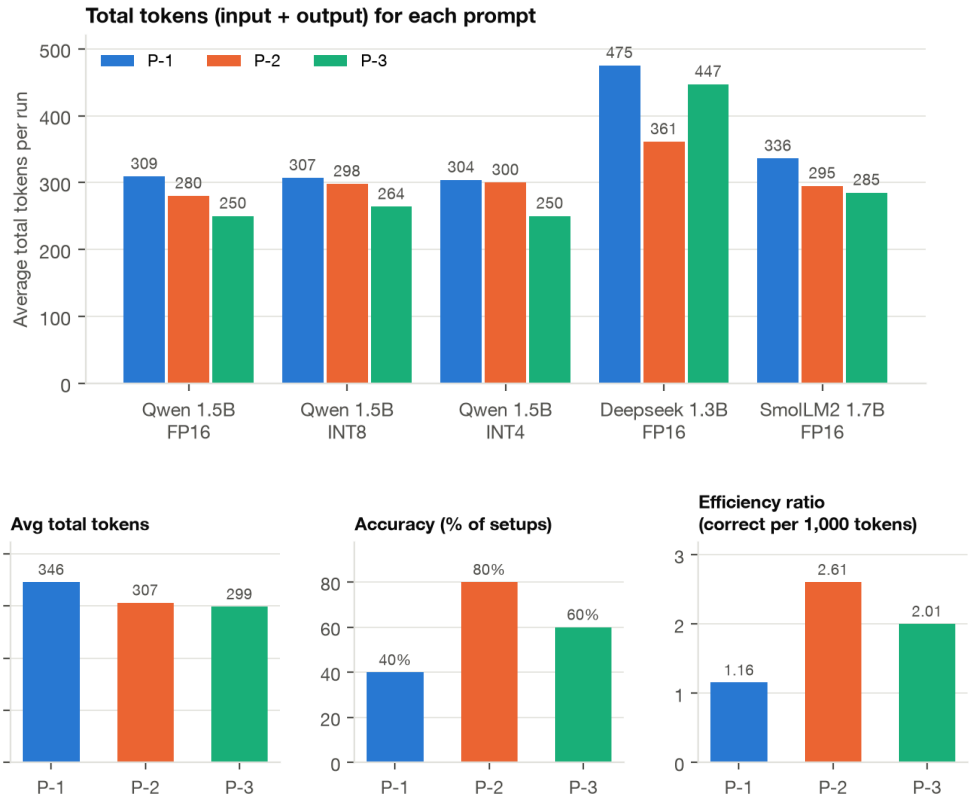
Which prompt used the fewest tokens? P-3 averaged 299 tokens per run, against 307 for P-2 and 346 for P-1. P-3 was the cheapest prompt for four of the five model setups, mostly because its input is about 43 tokens shorter than P-1.

The exception was Deepseek-Coder, which added explanation around the code with P-3 (447 tokens against 361 with P-2). A shorter prompt does not always give a shorter answer.

Which prompt was the most efficient? P-2 scored 2.61, against 2.01 for P-3 and 1.16 for P-1. It used only 8 more tokens than P-3 but was correct in 80% of setups, against 60%.

P-3 was sometimes too vague: Qwen returned the original docstring without Args or Returns sections. With P-1, Qwen put the docstring above the function instead of inside it.

Takeaway: tokens spent on a wrong answer are wasted, so the shortest prompt is not always the greenest.



3.2 Impact of Quantization & Hardware

> **Best quantization level: FP16 (unquantized)**, the fastest with no loss in accuracy. **Local vs cloud:** similar speed, but the laptop likely used 4–8× less energy per docstring.

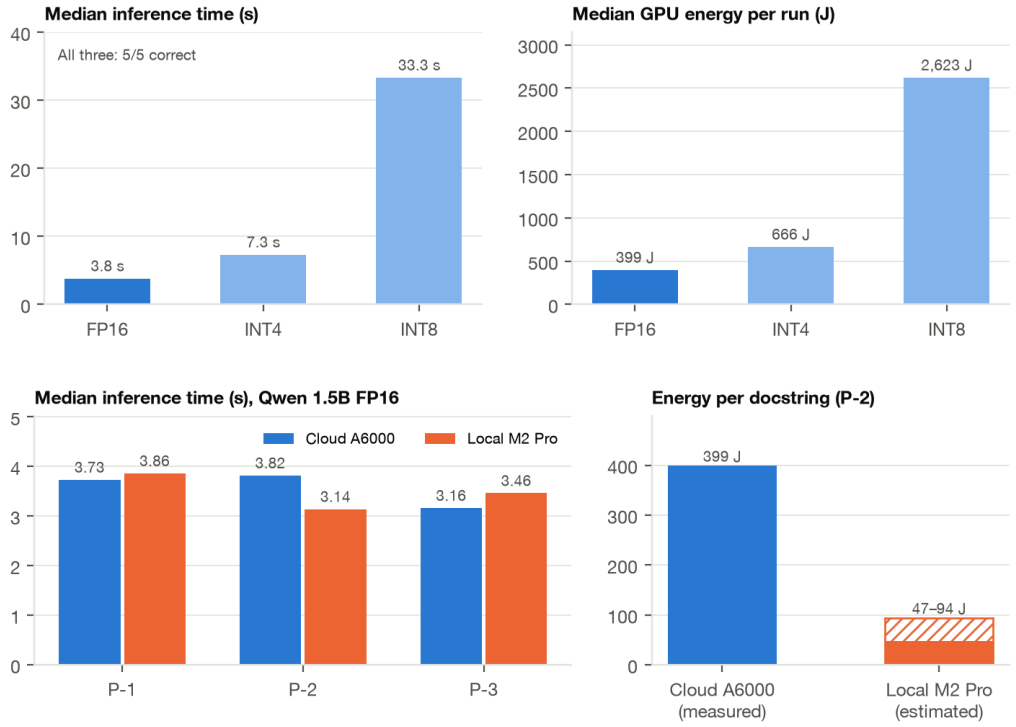
Quantization trade-off (Qwen 1.5B, prompt P-2). All three versions were correct in every run, but speed and energy changed:

- **FP16:** 3.8 s and 399 J per run
- **INT4:** 7.3 s and 666 J, about 2× slower
- **INT8:** 33.4 s and 2,623 J, about 9× slower

The model is small (about 3 GB) and the GPU has 48 GB, so memory was never the bottleneck. bitsandbytes unpacks the compressed weights during every calculation, which adds work. Quantization saved memory, not time or energy.

Local M2 Pro vs cloud A6000. Both produced exactly the same text at similar speed (P-2: 3.14 s on the laptop, 3.82 s on the GPU). The A6000 used about 400 J per docstring (about 105 W).

Laptop power could not be measured without administrator access, but a typical M2 Pro draws 15–30 W under this load, or roughly 47–94 J per docstring. For single requests to a small model, a large GPU mostly sits idle.



3.3 Model Robustness and Determinism

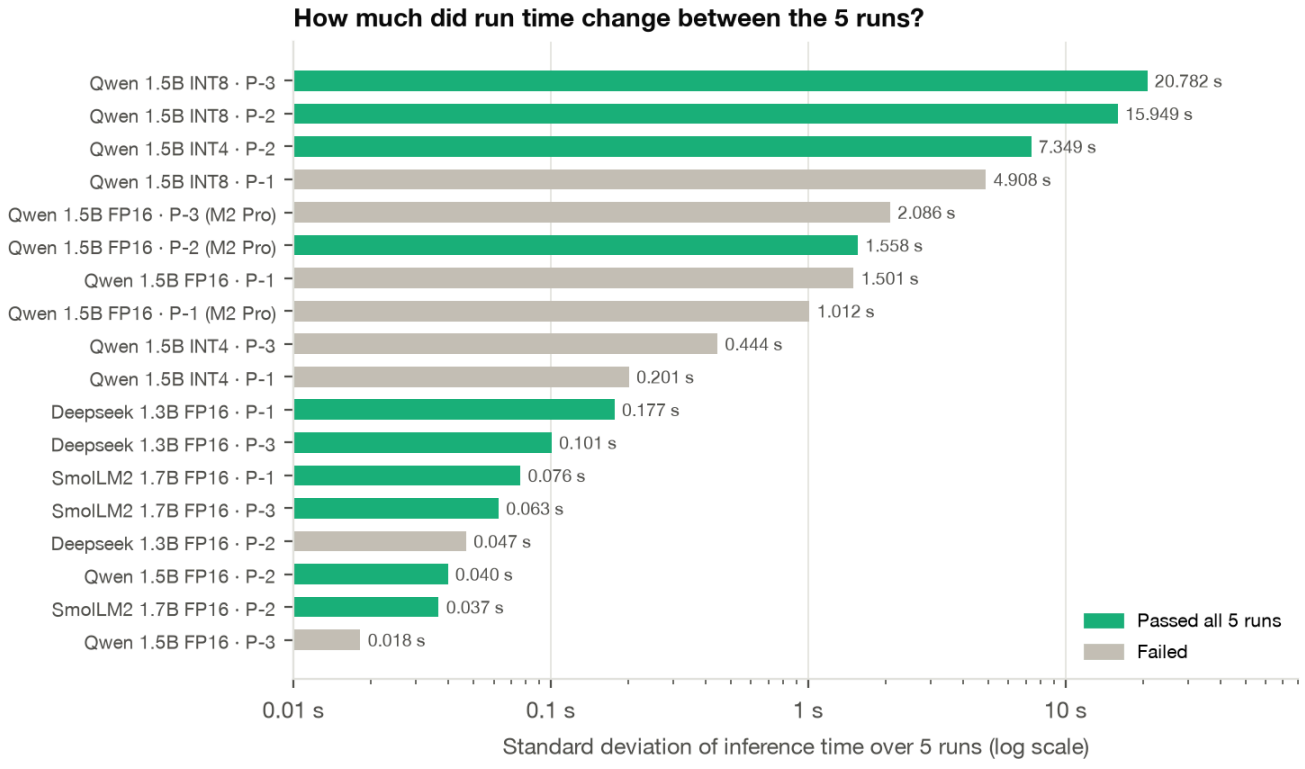
> **Most deterministic: Qwen 1.5B · FP16 · P-3** (time SD 0.018 s). Among passing combinations: **SmolLM2 · FP16 · P-2** (0.037 s) and **Qwen 1.5B · FP16 · P-2** (0.040 s).

Variance in output tokens (T_out). T_out had zero variance in all 18 combinations. With greedy decoding, each model wrote identical text in all 5 runs, so pass/fail never changed.

Variance in inference time. This is where the runs differed:

- **FP16 on the GPU was steady:** most combinations varied by under 0.1 s.
- **The quantized versions were the least steady.** Some INT8 runs jumped from 19 s to over 60 s (SD up to 20.8 s). The shared cloud machine was busy, and the quantized code relies more on the CPU.
- **The laptop varied by 24–48%** because it was in use for other work.

Takeaway: outputs are fully repeatable, but timing depends on machine load, so medians are safer than means.



3.4 Overall Green Recommendation

> **Recommended combination: Qwen2.5-Coder-1.5B-Instruct · FP16 (unquantized) · prompt P-2.**

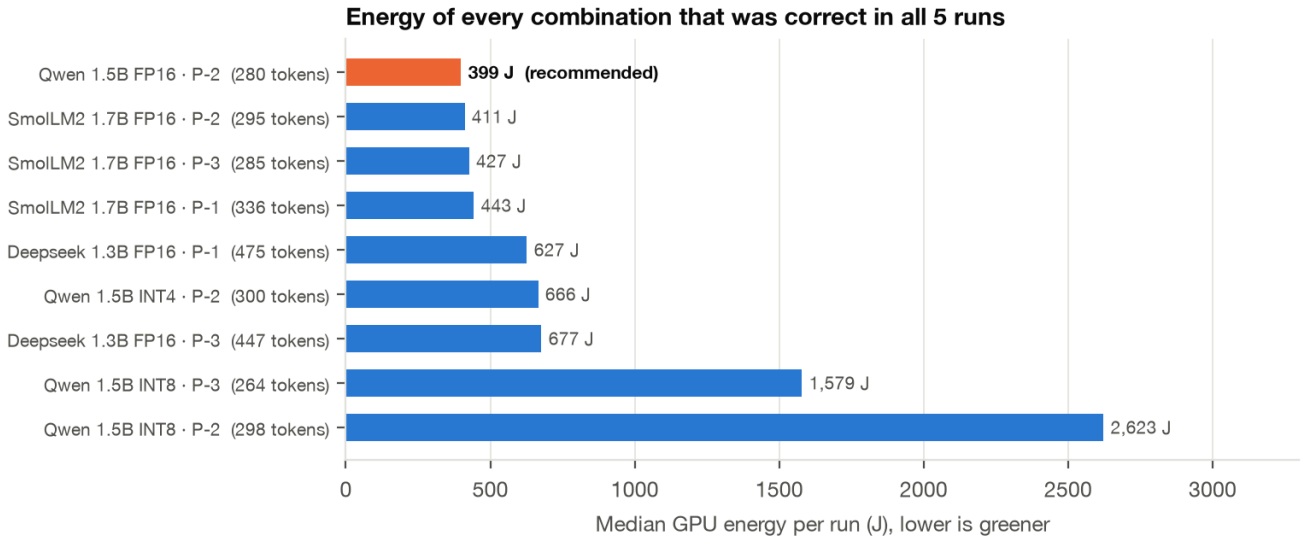
Why this combination:

- **Accuracy:** correct in 5 out of 5 runs, on both the cloud GPU and the laptop.
- **Fewest tokens:** 280, the lowest of any combination that was always correct.
- **Lowest energy:** 399 J per run, the lowest of any always-correct combination on the GPU.
- **Highest practical efficiency ratio:** 3.57 correct docstrings per 1,000 tokens.
- **Very stable:** run time varied by only 0.04 s.

Close alternatives:

- **SmolLM2 · FP16 · P-2** is a strong second choice: 295 tokens, 3.5 s and 411 J.
- **Qwen · INT8 · P-3** has a slightly higher efficiency ratio (3.79) but is not a good choice. It used about four times the energy (1,579 J) and took 19 s per run.

For the greenest result, run it on a local laptop: same answer, similar speed, a fraction of the energy.



Appendix A: Prompt Strings

`{code}` is replaced with the `below_zero` function exactly as given in the assignment. Each prompt was sent as a single user message through the model's chat template.

P-1 (Baseline, verbose)

```
Please act as an expert Python software engineer. Given the function below, I would
appreciate it if you could generate a complete and professional Google-style docstring.
The docstring should not include any extra commentary, only the docstring itself and the
original function. Thank you.

{code}
```

P-2 (Concise)

```
Generate Google-style docstring for the following Python function. Only output the
docstring and function code.

{code}
```

P-3 (Ultra-concise / constrained)

```
Python Docstring: {code}. Output only the new code.
```

Submitted files

File	Contents
`results/experimental_matrix.xlsx`	All 90 runs plus summary sheets
`results/experimental_matrix_all.csv`	All 90 runs as CSV
`results/*_outputs.jsonl`	The full generated text of every run, used for the manual P/F check
`prompts.md`	The three prompt strings and generation settings
`run_experiments.py`, `analyze.py`, `report_figures.py`	Code to run the experiments, summarise them, and draw the charts